Instituicao de Ensino Demo

Curso: Engenharia de Software

4o Semestre

Material de Estudo

Testes de Software

---

1. INTRODUCAO

Testes de software sao essenciais para garantir que o codigo funciona corretamente e continua funcionando apos mudancas. Esta disciplina ensina desde testes unitarios ate testes end-to-end, passando por TDD, BDD e automacao. Empresas de tecnologia valorizam profissionais que escrevem testes — e uma habilidade que diferencia engenheiros juniores de seniors.

Ementa: Fundamentos de testes. Testes unitarios, integracao e E2E. TDD e BDD. Automacao de testes. Testes de performance e seguranca. Cobertura de codigo.

2. CONCEITOS FUNDAMENTAIS

A seguir, apresentamos os conceitos-chave desta disciplina com explicacoes didaticas e exemplos praticos para facilitar o aprendizado.

2.1 Piramide de Testes e TDD

A piramide de testes sugere: muitos testes unitarios (rapidos, baratos), alguns testes de integracao e poucos testes E2E (lentos, caros). TDD (Test-Driven Development) inverte o fluxo: primeiro escreve o teste, depois o codigo que faz o teste passar, e por fim refatora. Ciclo: Red -> Green -> Refactor.

Exemplo:

```
TDD em Python com pytest:

# 1. RED - escreve teste que falha
def test_soma():
assert soma(2, 3) == 5

# 2. GREEN - implementa o minimo
def soma(a, b):
return a + b

# 3. REFACTOR - melhora o codigo
# (neste caso simples, nada a refatorar)
```

2.2 Testes de Integracao e E2E

Testes de integracao verificam que componentes funcionam juntos (ex: API + banco). Testes E2E (end-to-end) simulam o usuario real interagindo com o sistema completo. Mocks e stubs substituem dependencias externas em testes de integracao. Ferramentas: Cypress e Playwright para E2E, pytest-mock para mocks.

Exemplo:

```
Teste de integracao (API + banco):
def test_criar_aluno(client, db):
response = client.post("/api/alunos",
json={"nome": "Ana", "curso": "ES"})
assert response.status_code == 201
aluno = db.query(Aluno).first()
assert aluno.nome == "Ana"
```

3. RESUMO DOS PRINCIPAIS TOPICOS

O conteudo programatico da disciplina esta organizado nas seguintes unidades:

- Unidade 1: Fundamentos de testes - niveis, tipos, estrategias
- Unidade 2: Testes unitarios - JUnit, pytest, Jest
- Unidade 3: TDD (Test-Driven Development) na pratica
- Unidade 4: Testes de integracao e mocks
- Unidade 5: BDD (Behavior-Driven Development) e Gherkin
- Unidade 6: Testes E2E - Selenium, Cypress, Playwright
- Unidade 7: Testes de performance (JMeter) e seguranca (OWASP ZAP)
- Unidade 8: Cobertura de codigo, CI/CD e qualidade continua

Cada unidade aborda um aspecto fundamental da disciplina, construindo progressivamente o conhecimento necessario para dominar o tema. Recomenda-se estudar as unidades na ordem apresentada.

4. EXERCICIOS DE FIXACAO

Resolva os exercicios abaixo para verificar seu entendimento dos conceitos apresentados.

Questao 1 (Multipla Escolha)

No TDD, qual e a ordem correta do ciclo?

a) Green -> Red -> Refactor
b) Red -> Green -> Refactor
c) Refactor -> Red -> Green
d) Red -> Refactor -> Green

Questao 2 (Dissertativa)

Explique a piramide de testes e por que testes unitarios devem ser a maioria.

Questao 3 (Multipla Escolha)

Qual ferramenta e usada para testes E2E em aplicacoes web?

a) pytest
b) JUnit
c) Cypress
d) Mockito

5. DICAS DE ESTUDO E RECURSOS ADICIONAIS

Para maximizar seu aprendizado nesta disciplina, siga estas recomendacoes:

• Pratique TDD em projetos pessoais — comece com funcoes simples.
• Escreva testes ANTES de corrigir bugs — garante que o bug nao volta.
• Aprenda a usar mocks, mas nao abuse — prefira testes sem mock.
• Configure CI/CD para rodar testes automaticamente a cada commit.

Bibliografia Recomendada:

• BECK, K. TDD: Desenvolvimento Guiado por Testes. Bookman, 2010.
• ANICHE, M. Testes Automatizados de Software. Casa do Codigo, 2015.
• HUMBLE, J.; FARLEY, D. Entrega Continua. Bookman, 2014.

6. GLOSSARIO DE TERMOS IMPORTANTES

Consulte este glossario sempre que encontrar um termo desconhecido durante seus estudos.

- **Teste unitario**: testa uma unidade isolada de codigo (funcao, classe).
- **Teste de integracao**: testa a interacao entre componentes.
- **Teste E2E**: testa o sistema completo do ponto de vista do usuario.
- **TDD**: Test-Driven Development — testes antes do codigo.
- **BDD**: Behavior-Driven Development — testes baseados em comportamento.
- **Mock**: objeto falso que simula uma dependencia em testes.
- **Cobertura**: percentual do codigo exercitado pelos testes.